



Open FOAM

Thermal Simulation of a CPU Cooler: Modeling Heat Sources and Fans (Part 2)

Nicola Andreini, PhD

A practical LinkedIn series to simulate real-world engineering scenarios with OpenFOAM.

Focused on CFD for **thermal management**, **multiphase flow**, phase change phenomena , and **common tasks**.

Goal :

Impose velocity profile and create a patch

What you'll see:

- How to define patch on a surface
- How to model fan performance using characteristic curves in boundary conditions



If you find this content valuable : **share the post** and **leave a comment** with real-world cases, **you'd like to see covered**

- **Multi-region setup:** Define separate cellZones for fluid and solid domains (fluid region and processor in this case)
- **Boundary condition targeting:** Create faceSets/faceZones for patches (e.g., fan interfaces, porous zones, baffles)

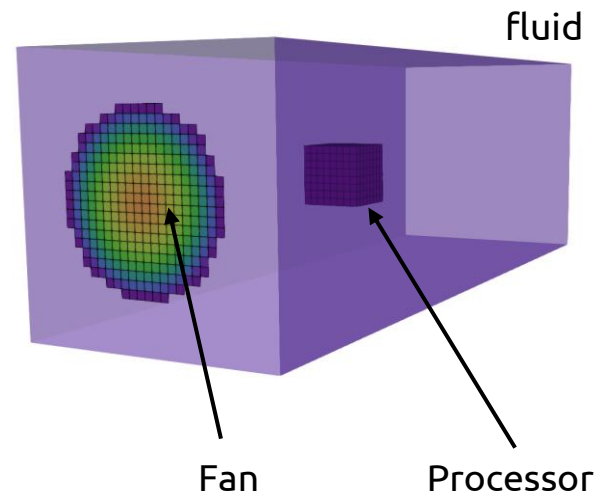
1

```
actions
{
    // SOLID REGION (processor)
    {
        name    solidCellSet;
        type    cellSet;
        action  new;
        source  boxToCell;
        box     (0.100 0.030 0.030) (0.120 0.050 0.050);
    }
    {
        name    solid;
        type    cellZoneSet;
        action  new;
        source  setToCellZone;
        set     solidCellSet;
    }

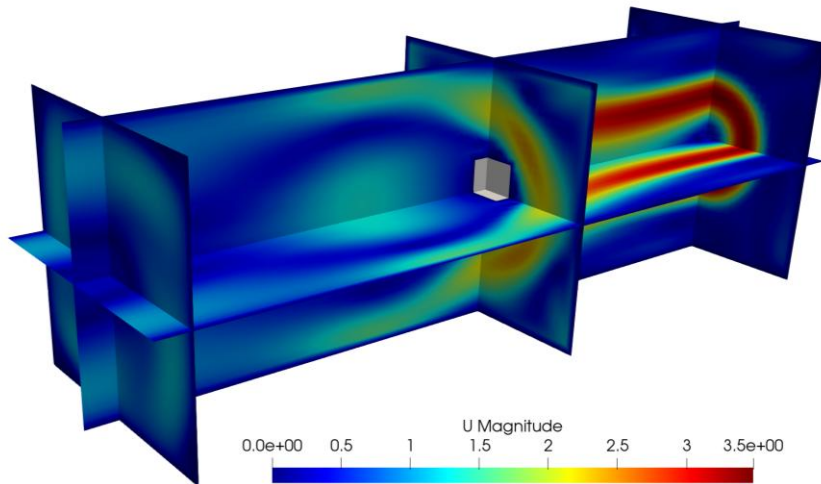
    // FAN PATCH
    {
        name    fanFaceSet;
        type    faceSet;
        action  new;
        source  cylinderToFace;
        sourceInfo
        {
            point1 (-0.001 0.040 0.040);
            point2 (0.001 0.040 0.040);
            radius  0.025;
        }
    }
}
```

2

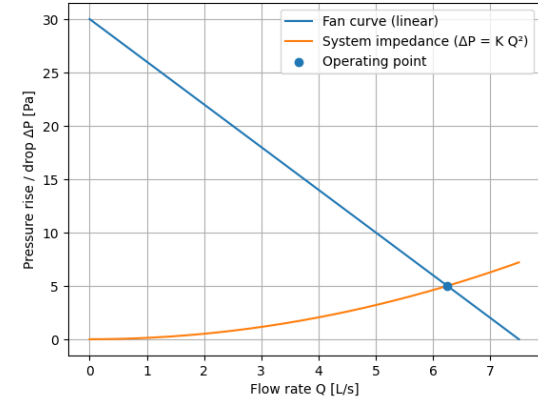
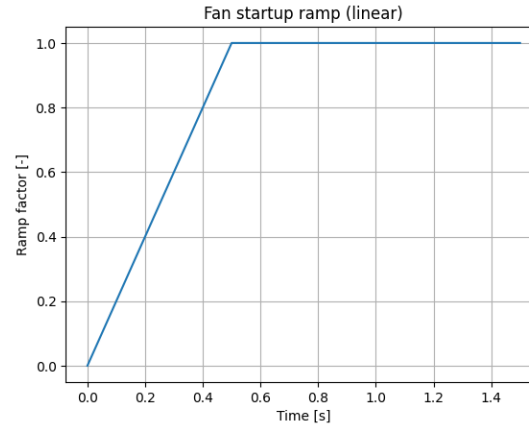
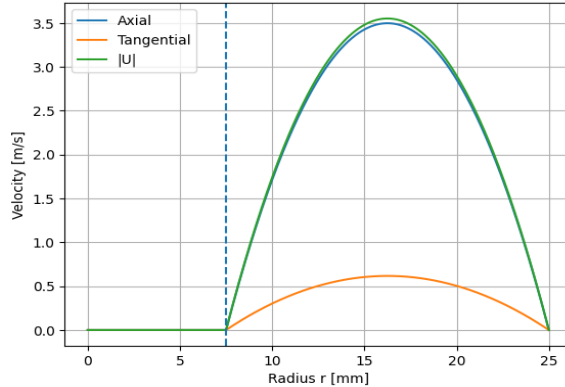
```
// FLUID REGION
{
    name    fluidCellSet;
    type    cellSet;
    action  new;
    source  boxToCell;
    box     (-1 -1 -1) (1 1 1); // All cells in domain
}
// Remove solid cells from fluid
{
    name    fluidCellSet;
    type    cellSet;
    action  delete;
    source  cellToCell;
    set     solidCellSet;
}
{
    name    fluid;
    type    cellZoneSet;
    action  new;
    source  setToCellZone;
    set     fluidCellSet;
}
);
```



```
pointSync false;  
  
patches  
(  
  {  
    name fan;  
    patchInfo  
    {  
      type patch;  
    }  
    constructFrom set;  
    set fanFaceSet;  
  }  
);
```



- **constructFrom** - Source for face selection: patches, faceSet, faceZone
- **patchInfo** - Defines new patch properties: patch, wall, symmetry, cyclic, cyclicAMI, etc.
- **set/zone** - Name of the faceSet/faceZone to convert into a patch



- **Predefined operating point:** the flow rate is computed a priori from a simplified fan curve + system loss, not solved dynamically by the CFD.
- **Simplified fan and system curves:** fan performance is assumed linear, and system pressure drop is treated as constant.
- **Fixed swirl angle:** swirl is added via a constant turning angle, not linked to torque/RPM/blade design.
- **Simple startup transient:** fan ramp-up is represented by a linear time ramp for numerical smoothness.



This example is **intentionally simplified for teaching**. The goal is not to deliver a highly accurate fan model, but to **provide the conceptual** tools to build one.

1

```

fan
{
    type            codedFixedValue;
    value            uniform (0 0 0);
    name             simpleFanEducational;

    code
    #{

        // Fan radius [meters]
        const scalar fanRadius = 0.025;
        // Fan center in the Y-Z plane
        const vector fanCenter(0, 0.040, 0.040);
        // Pi constant
        const scalar pi = constant::mathematical::pi;
        // Hub-to-radius ratio [-]
        const scalar hubRatio = 0.30; // 30% of radius is occupied by the hub
        // Hub radius [m]
        const scalar hubRadius = hubRatio * fanRadius;
        // Area of the ANNULUS where air flows [m^2]
        const scalar fanArea = pi * (pow(fanRadius,2) - pow(hubRadius,2));

        // Maximum flow rate (at zero pressure) [m^3/s]
        const scalar Q_max = 0.0075;
        // Maximum pressure rise (at zero flow) [Pa]
        const scalar dP_max = 30.0;
        // System pressure drop (CPU + heatsink) [Pa]
        const scalar dP_system = 5.0;
    }
}

```

- The Fan class uses **codedFixedValue** type with name `simpleFanEducational` for custom boundary condition implementation
- **Geometric parameters of the fan are defined:** radius (`fanRadius`), center in the Y-Z plane (`fanCenter`), and hub-to-radius ratio (`hubRatio = 0.3`)
- **The hub radius is calculated as a fraction of the total radius:** $hubRadius = hubRatio * fanRadius$
- The **annular area** for air flow is determined by the difference between areas:
 - $fanArea = \pi * (fanRadius^2 - hubRadius^2)$
- **Operating parameters are specified:** maximum flow rate at zero pressure (`Q_max`) and maximum pressure rise at zero flow (`dP_max`)

2

```
// Operating flow rate
scalar Q_operating = Q_max * (1.0 - dP_system / dP_max);
// Ensure it is positive
Q_operating = max(Q_operating, 0.001); // minimum 1 L/s
// Average velocity across the fan face [m/s]
const scalar U_mean = Q_operating / fanArea;

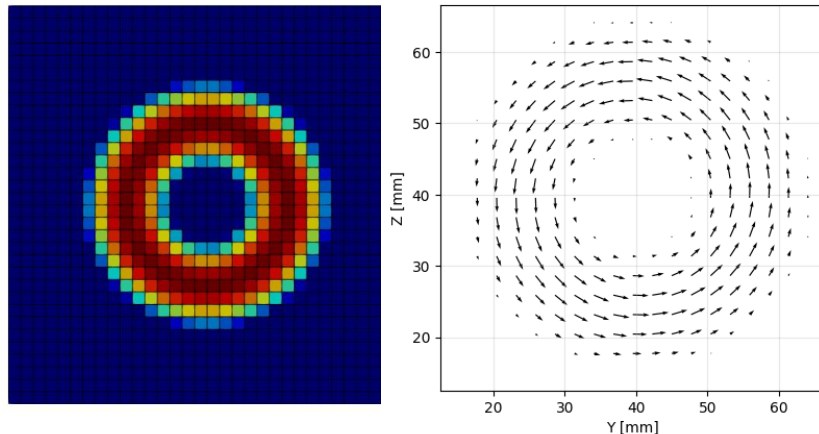
// Current simulation time [s]
const scalar t = this->db().time().value();
// Startup ramp duration [s]
const scalar t_ramp = 0.5; // half a second

scalar rampFactor = 1.0;
if (t < t_ramp)
{
    // Simple linear ramp
    rampFactor = t / t_ramp;
}

// Instantaneous velocity (with ramp applied)
const scalar U_instantaneous = rampFactor * U_mean;

// Swirl angle [degrees]
const scalar swirlAngle_deg = 10.0;
const scalar swirlAngle = swirlAngle_deg * pi / 180.0;
```

- The **startup ramp** is used for accelerating the fan from rest.
- **Average velocity** is calculated as: $U_{mean} = Q_{operating} / fanArea$ with safety check $Q_{operating} = \max(Q_{operating}, 0.001)$
- Current **simulation time** is retrieved using:
 $this->db().time().value()$



3

```
// Print information every 50 iterations to monitor
if (Pstream::master())
{
    if (this->db().time().timeIndex() % 50 == 0)
    {
        Info << "\n";
        Info << "==== FAN MODEL =====< < endl;
        Info << "   Time:           << t << " s" << endl;
        Info << "   Ramp:           << rampFactor * 100 << " %" << endl;
        Info << "   Flow rate:        << Q_operating * 1000 << " L/s" << endl;
        Info << "   U mean:          << U_instantaneous << " m/s" << endl;
        Info << "   Swirl:           << swirlAngle_deg << " deg" << endl;
        Info << "=====< < endl;
        Info << "\n";
    }
}

// Face centers of the patch
const vectorField& faceCenters = patch().Cf();
// Velocity field to fill
vectorField& field = *this;

forAll(faceCenters, i)
{
    // Coordinates relative to center
    const scalar dy = faceCenters[i].y() - fanCenter.y();
    const scalar dz = faceCenters[i].z() - fanCenter.z();
    // Distance from center (radius)
    const scalar r = Foam::sqrt(dy*dy + dz*dz);

    if (r < hubRadius)
    {
        // Inside the hub: zero velocity
        field[i] = vector(0, 0, 0);
        continue; // skip to next face
    }
}
```

- Debug information is printed every 50 iterations to track simulation progress.
- The condition *if (this->db().time().timeIndex() % 50 == 0)* checks if the code is running on the master processor and if the iteration count is divisible by 50 **print info to the console**
- **Face centers** are accessed with: *const vectorField& faceCenters = patch().Cf;*
- The **loop iterates over all faces**: *forAll(faceCenters, i)* where *i* is the face index
- Local **Y and Z coordinates** are computed: *const scalar cy = faceCenters[i].y() - fanCenter.y();* and *const scalar cz = faceCenters[i].z() - fanCenter.z();*
- **Hub region** is identified with: *if (r < hubRadius)* then *field[i] = vector(0, 0, 0);* and *continue;* to **skip to next face**

4

```
scalar r_star = (r - hubRadius) / (fanRadius - hubRadius);
r_star = max(0.0, min(r_star, 1.0));

// Parabolic profile in the annulus
scalar U_axial = 4.0 * U_instantaneous * r_star * (1.0 - r_star);
U_axial = max(U_axial, 0.0);
scalar U_tangential = U_axial * Foam::tan(swirlAngle);

vector tangentDir(0, 0, 0);
if (r > 1e-6)
{
    tangentDir = vector(0, -dz/r, dy/r);
}
// Axial direction vector
vector axialDir(1, 0, 0);
// Total velocity
field[i] = U_axial * axialDir
          + U_tangential * tangentDir;
}
#);
}
```

- A **normalized radial position** is calculated: $scalar\ r_star = (r - hubRadius) / (fanRadius - hubRadius)$;
- The profile uses $r_star = max(0.0, min(r_star, 1.0))$; to **clamp values between 0 and 1**, ensuring numerical stability
- A **parabolic velocity distribution** is applied: $scalar\ U_axial = 4.0 * U_instantaneous * r_star * (1.0 - r_star)$;
- The **complete velocity vector** combines both components: $field[i] = U_axial * axialDir + U_tangential * tangentDir$;

Complete simulation case with all available at:

https://github.com/nicolaandreini/OpenFOAM_Notes_in_ThermoFluidDynamics

